CprE 3810: Computer Organization and Assembly-Level Programming
Project Part 2 Report

Team Members: Graeme Anderson
Colten Stevens

Project Teams Group #: C_01

Refer to the highlighted language in the project 1 instruction for the context of the following questions.

[1.a] Come up with a global list of the datapath values and control signals that are required during each pipeline stage.

IF:

None

ID:

- Instruction
- PC
- Funct 3
- JumpWReg
- Jump
- Branch

EX:

- RS1 value
- RS2 value
- PC + 4
- Immediate
- Instruction
- ALUsrc
- ALUctrl
- AndLink
- Shift
- FlagMux
- FlagOrNFlag

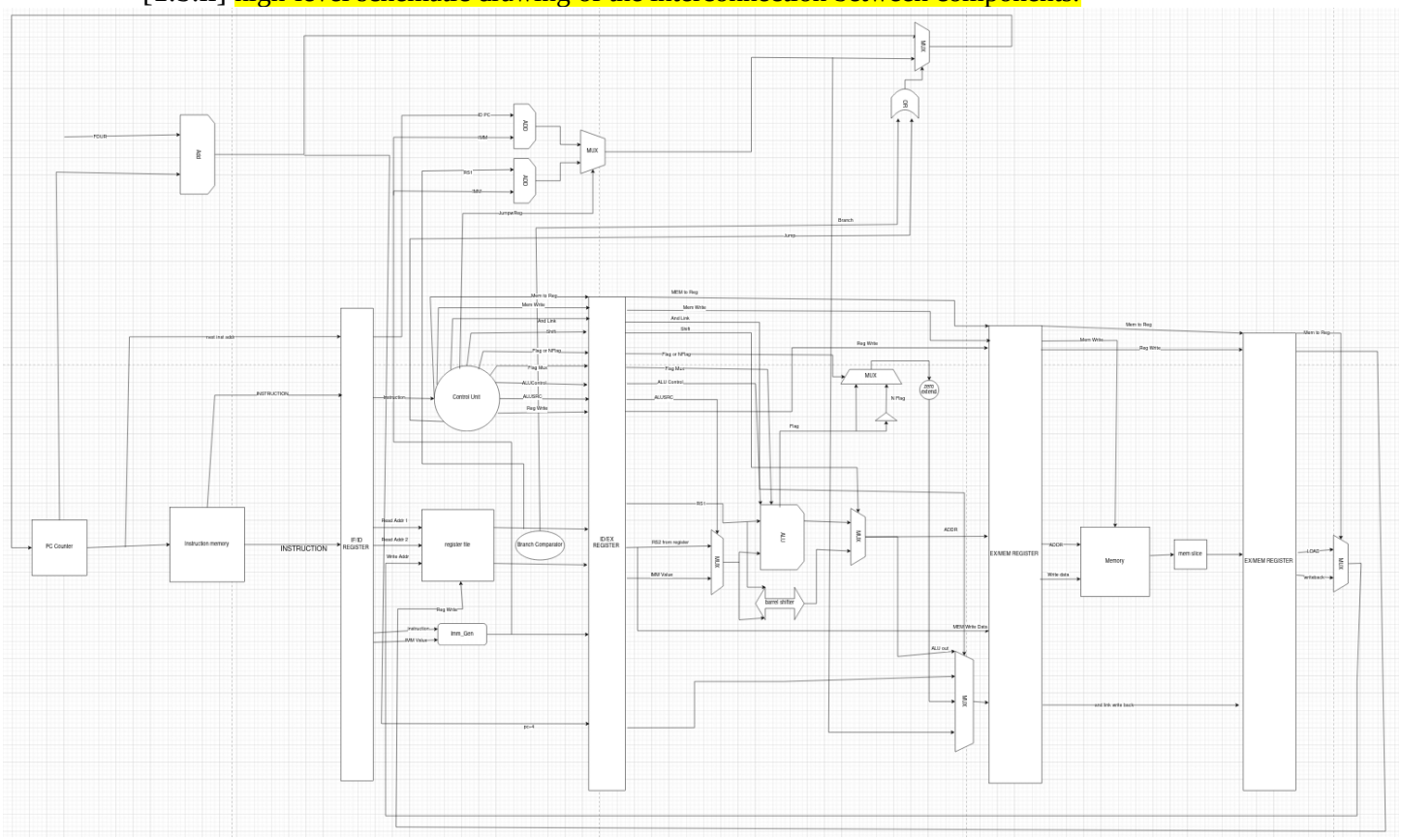
MEM:

- EX Stage Result
- Instruction
- RS2 value
- Funct 3
- MemWrite

WB:

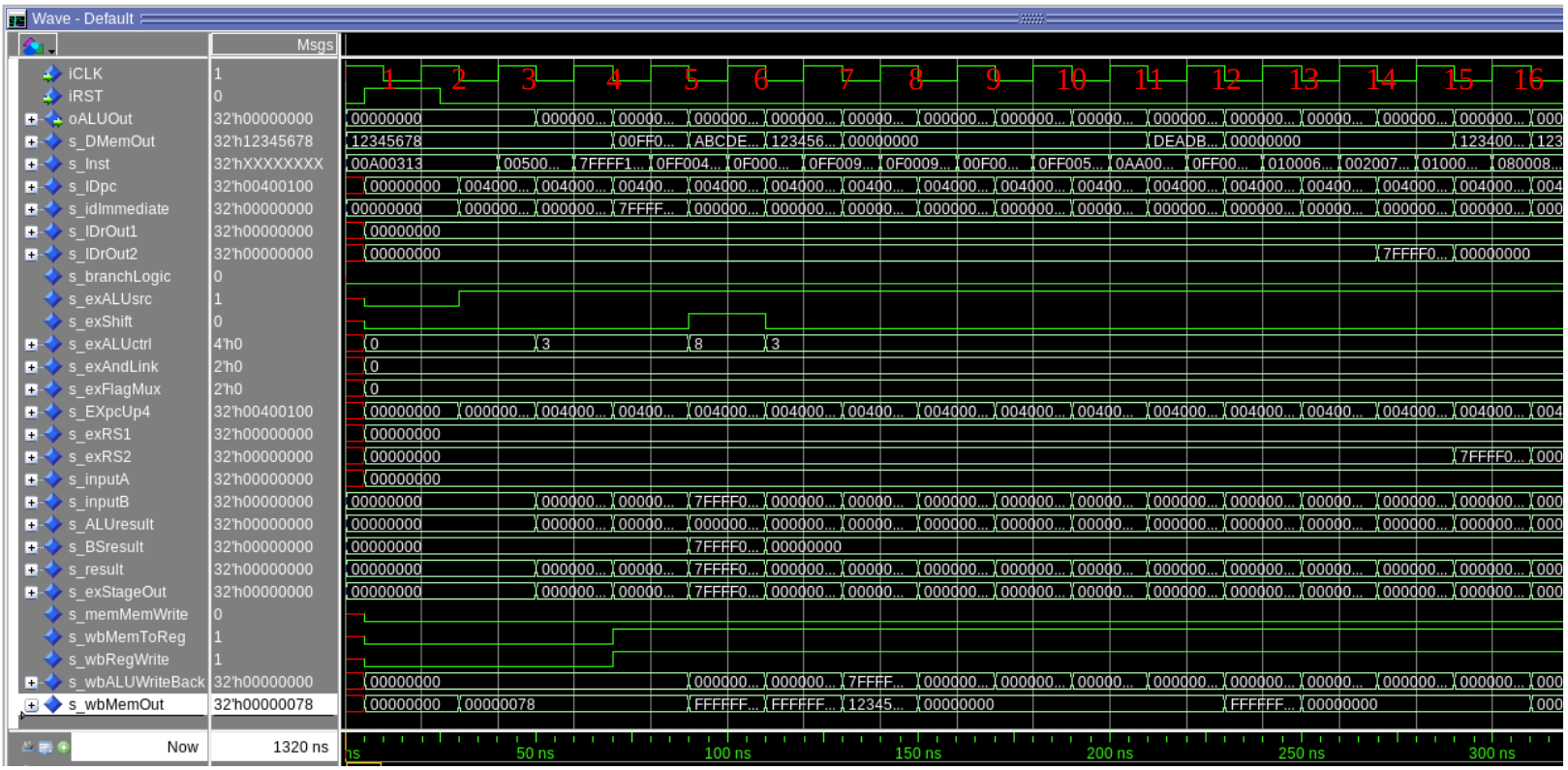
- Instruction
- Memory Output
- Ex Stage Result
- RegWrite
- Halt

[1.b.ii] high-level schematic drawing of the interconnection between components.



[1.c.i] include an annotated waveform in your writeup and provide a short discussion of result correctness.

Base Tests

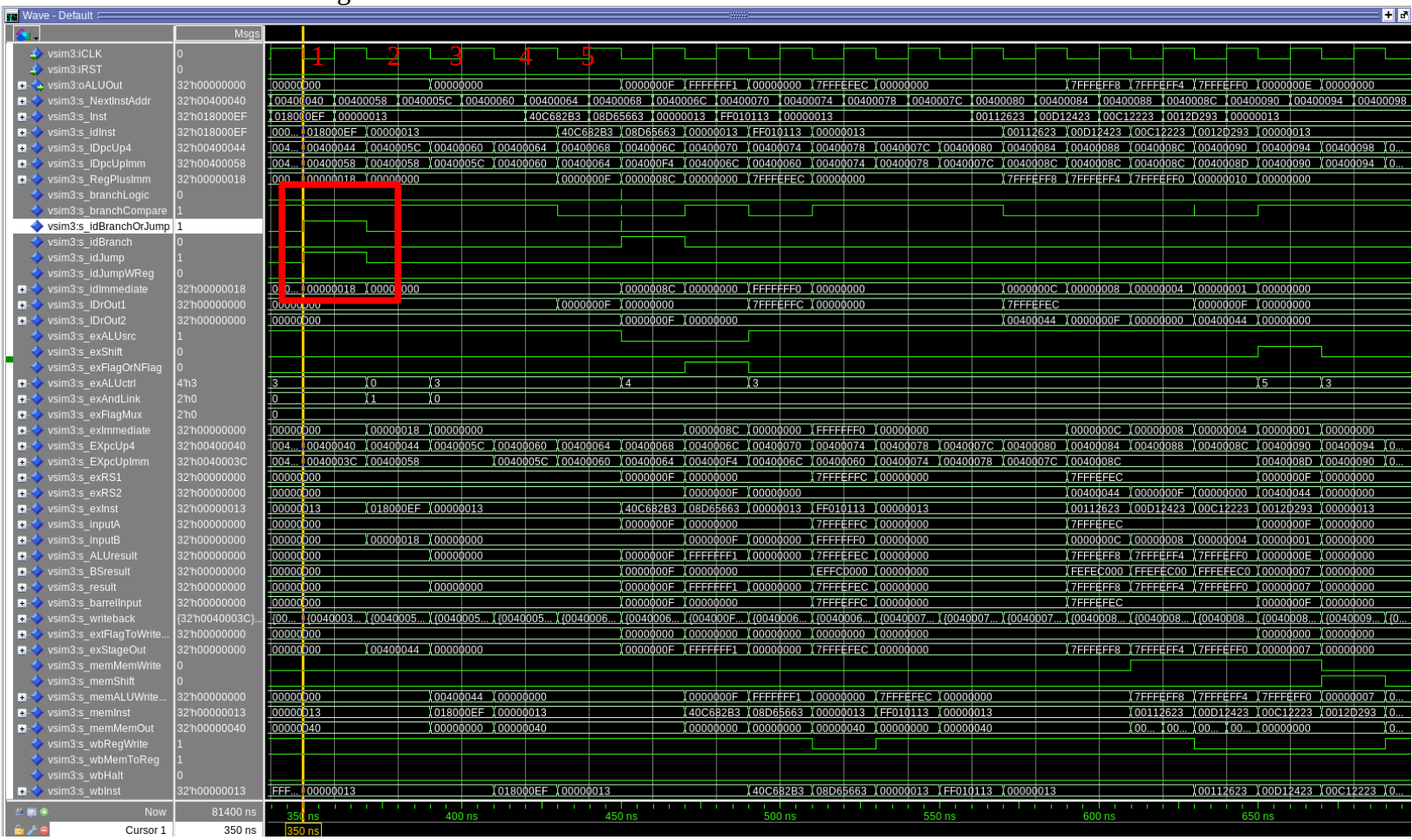


For the first 3 cycles, you can see the initial propogation through the stages, as the zeros turn into values in the middle of cycle 4, when the first instruction hits the writeback stage.

These base tests are simplistic in their design, with the majority of these first 16 instructions being various ADDI instructions. These instructions passing, along with their control signals verify that the pipeline registers propagate the necessary signals through the pipeline.

[1.c.ii] Include an annotated waveform in your writeup of two iterations or recursions of these programs executing correctly and provide a short discussion of result correctness. In your waveform and annotation, provide 3 different examples (at least one data-flow and one control-flow) of where you did not have to use the maximum number of NOPs.

Merge Sort



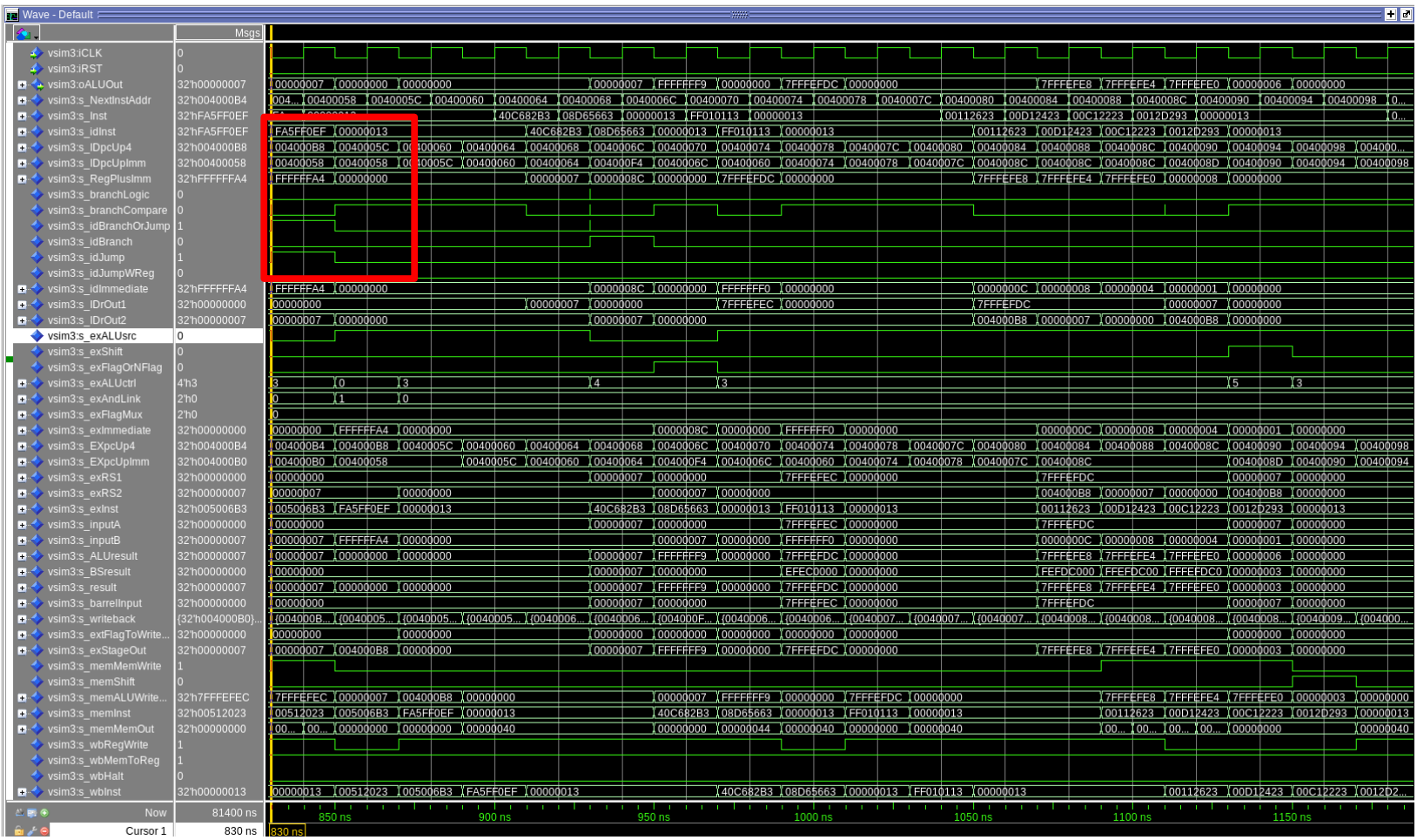
Iteration 1

The first rectangle highlights the signals denoting our branch logic jumping to high, enabling a PC update from 0040 to 0058 (shortened addr.) This shows the first jump into the labelled section ‘mergeSort’.

Through the waveform, you can see directly following the jump, 3 NOP instructions follow, allowing any instructions before the jump to fully execute, preventing the jump from causing a data hazard.

```
main:
    lui  a1, 0x10010
    lui  a0, 0x10010
    nop
    nop
    addi a1, a1, 0x040
    addi a0, a0, 0x000
    nop
```

This snippet shows an optimization of the assembly code, where instead of using 2 consecutive LASW instructions, we expanded the instructions and ordered them to reduce NOPs from the necessary 3.



Iteration 2

This second iteration of the mergeSort section shows the same 3 NOPs and the same jump pattern under the first outline.

The next instructions follow through the same pattern of the assembly code in the mergeSort section.

These two code snippets provide two more cases of avoiding data hazards and control hazards without maximum NOPs. The left image shows slli t0, s5, 2 being spaced out from add t0, t0, a0 by 3 instructions to prevent NOP usage between those instructions.
On the right, our Jumps and Branches only require a single NOP following each instruction due to our implementation of Non-continous PC updating instructions. Since we complete them in the Instruction Decode stage, we must have a single NOP behind each in the SW pipeline, which is significantly better than other implementations.

```
sub  s3, s6, s5 # Setup for n1
slli t0, s5, 2  # Base Address Precomputing
sub  s4, s7, s6 # s4 = n2
addi s0, x0, 0  # Copy left subarray (s0 = i = 0)
addi s3, s3, 1  # s3 = n1
add  t0, t0, a0 # t0 = &arr[left]
nop
nop
```

```
addi t1, t1, 4
addi t2, t2, 4

j copyLeft
nop
```

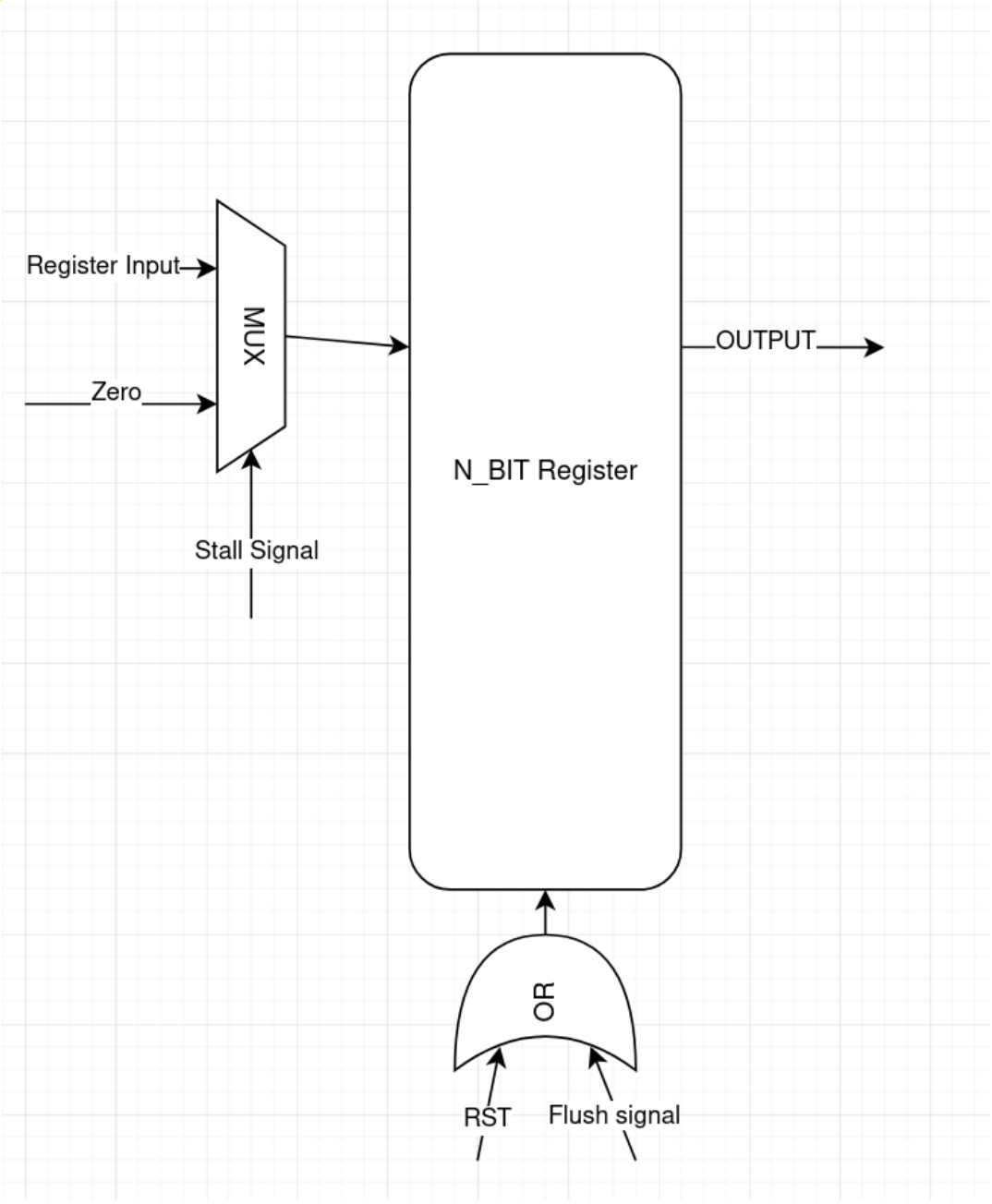
[1.d] report the maximum frequency your software-scheduled pipelined processor can run at and determine what your critical path is (specify each module/entity/component that this path goes through).

31.59 MHz

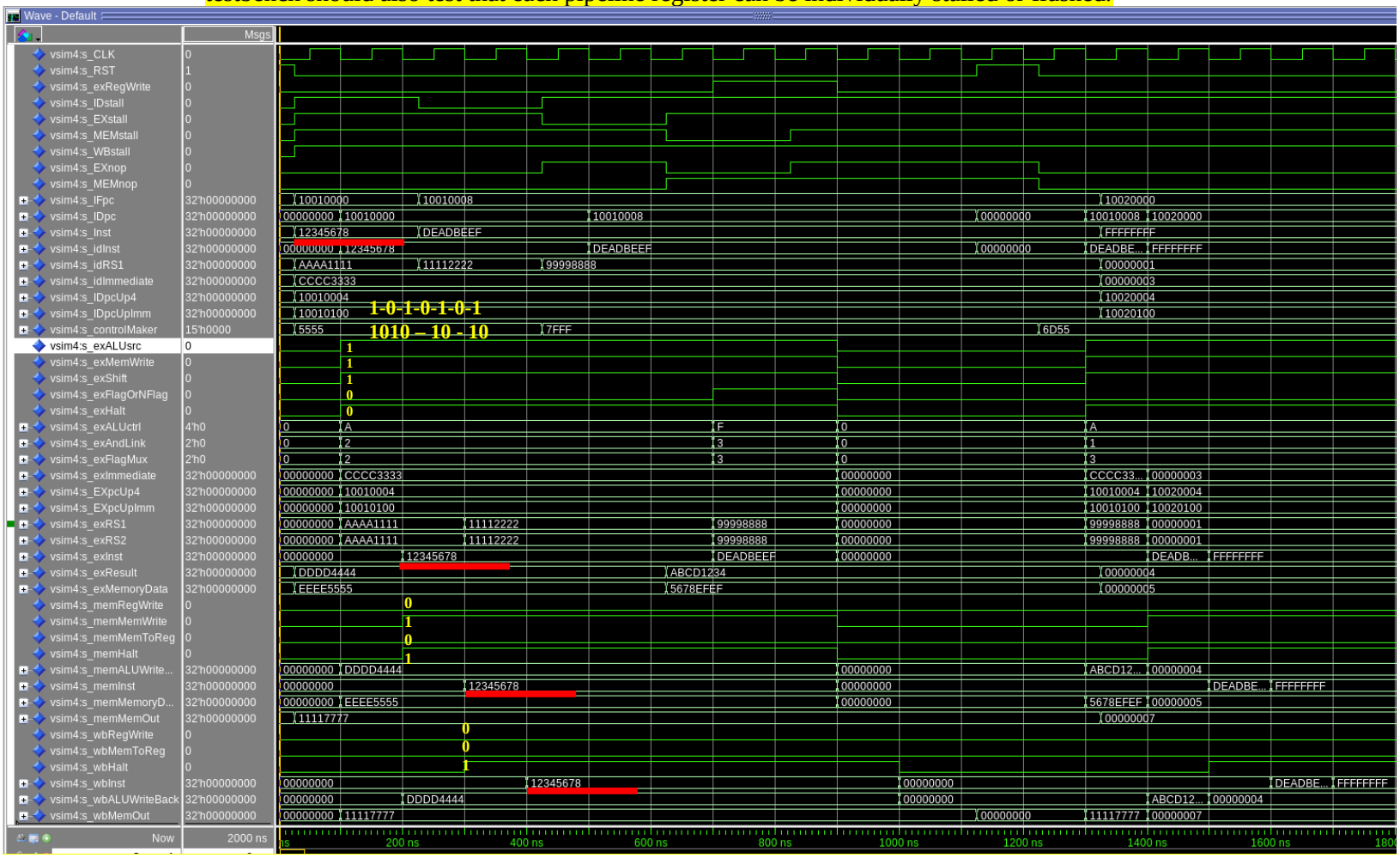
Critical Path:
Imem → Register File → Adder For JALR → FetchUnit (Update PC)

HARDWARE PIPELINE

[2.a.ii] Draw a simple schematic showing how you could implement stalling and flushing operations given an ideal N-bit register.



[2.a.iii] Create a testbench that instantiates all four of the registers in a single design. Show that values that are stored in the initial IF/ID register are available as expected four cycles later, and that new values can be inserted into the pipeline every single cycle. Most importantly, this testbench should also test that each pipeline register can be individually stalled or flushed.



This waveform shows our 4 pipeline registers passing signals through as you can see clearly with the red highlights.

The control maker signal was created to concatenate the control signals into a single easier to write to input. It operated as such:
First 7 bits are the 7 single bit control signals
(0)ALUsrc, (1)RegWrite, (2)MemWrite, (3)MemToReg, (4)Shift, (5)FlagOrNFlag, and (6)Halt
The next 4 bits is (10-7)ALUCtrl
With the final 4 bits split 2 & 2, with 2 bits (12-11)AndLink and 2 bits (14-13) FlagMux in that order.

I mostly only included the signals at the stage they were used in order to reduce clutter on the waveform. The yellow numbers indicate the values of the respective control signals which are also highlighted in yellow as they appear in the waveform.

[2.b.i] list which instructions produce values, and what signals (i.e., bus names) in the pipeline these correspond to.
[2.b.ii] List which of these same instructions consume values, and what signals in the pipeline these correspond to.

Instruction	IF Stage		ID Stage		EX Stage			MEM Stage		WB Stage	
	pc read	pc new value	read signal 1	Read signal 2	Read signal 1	read signal 2	write signal	read signal	Write signal	read signal	write signal
addi	input_pc	pcUp4	RS1	IMM	RS1	IMM	S_ALUresult = RS1 + IMM	s_memALUWriteBack	s_memALUWriteBack	s_wbALUWriteBack	s_RegWrData
add	input_pc	pcUp4	RS1	RS2	RS1	RS2	S_ALUresult = RS1 +RS2	s_memALUWriteBack	s_memALUWriteBack	s_wbALUWriteBack	s_RegWrData
and	input_pc	pcUp4	RS1	RS2	RS1	RS2	S_ALUresult = RS1 && RS2	s_memALUWriteBack	s_memALUWriteBack	s_wbALUWriteBack	s_RegWrData
andi	input_pc	pcUp4	RS1	IMM	RS1	IMM	S_ALUresult = RS1 && IMM	s_memALUWriteBack	s_memALUWriteBack	s_wbALUWriteBack	s_RegWrData
lui	input_pc	pcUp4	RS1	IMM	RS1	IMM	S_ALUresult	s_memALUWriteBack	s_memMemOut	s_wbMemOut	s_RegWrData
lw	input_pc	pcUp4	RS1	IMM	RS1	IMM	S_ALUresult = RS1 && IMM : RS2 passed to next stage	s_memALUWriteBack	s_memMemOut	s_wbMemOut	s_RegWrData
xor	input_pc	pcUp4	RS1	RS2	RS1	RS2	S_ALUresult = RS1 ^ RS2	s_memALUWriteBack	s_memALUWriteBack	s_wbALUWriteBack	s_RegWrData
xori	input_pc	pcUp4	RS1	IMM	RS1	IMM	S_ALUresult = RS1 ^ IMM	s_memALUWriteBack	s_memALUWriteBack	s_wbALUWriteBack	s_RegWrData
or	input_pc	pcUp4	RS1	RS2	RS1	RS2	S_ALUresult = RS1 || RS2	s_memALUWriteBack	s_memALUWriteBack	s_wbALUWriteBack	s_RegWrData
ori	input_pc	pcUp4	RS1	IMM	RS1	IMM	S_ALUresult = RS1 || RS2	s_memALUWriteBack	s_memALUWriteBack	s_wbALUWriteBack	s_RegWrData
slt	input_pc	pcUp4	RS1	RS2	RS1	RS2	S_ALUresult = (RS1<RS2)	s_memALUWriteBack	s_memALUWriteBack	s_wbALUWriteBack	s_RegWrData
slti	input_pc	pcUp4	RS1	IMM	RS1	IMM	S_ALUresult = (RS1<IMM)	s_memALUWriteBack	s_memALUWriteBack	s_wbALUWriteBack	s_RegWrData
sltiu	input_pc	pcUp4	RS1	IMM	RS1	IMM	S_ALUresult = (RS1<IMM) UNSIGNED	s_memALUWriteBack	s_memALUWriteBack	s_wbALUWriteBack	s_RegWrData
sll	input_pc	pcUp4	RS1	RS2	RS1	RS2	S_Bsresult = RS1 << RS2	s_memALUWriteBack	s_memALUWriteBack	s_wbALUWriteBack	s_RegWrData
srl	input_pc	pcUp4	RS1	RS2	RS1	RS2	S_Bsresult = RS1 >> RS2 left bits replaced by 0's	s_memALUWriteBack	s_memALUWriteBack	s_wbALUWriteBack	s_RegWrData
sra	input_pc	pcUp4	RS1	RS2	RS1	RS2	S_Bsresult = RS1 >> RS2 left bits replaced by sign bit	s_memALUWriteBack	s_memALUWriteBack	s_wbALUWriteBack	s_RegWrData
sw	input_pc	pcUp4	RS1	IMM	RS1	IMM	commputes adress RS1 + imm and stores RS2	s_memMemWrite	n/a	n/a	n/a
sub	input_pc	pcUp4	RS1	RS2	RS1	RS2	S_ALUresult = RS1 – RS2	s_memALUWriteBack	s_memALUWriteBack	s_wbALUWriteBack	s_RegWrData
beq	input_pc	pcUp4 or pc+imm	RS1	RS2	RS1	RS2	n/a	n/a	n/a	n/a	n/a
bne	input_pc	pcUp4 or pc+imm	RS1	RS2	RS1	RS2	n/a	n/a	n/a	n/a	n/a
blt	input_pc	pcUp4 or pc+imm	RS1	RS2	RS1	RS2	n/a	n/a	n/a	n/a	n/a
bge	input_pc	pcUp4 or pc+imm	RS1	RS2	RS1	RS2	n/a	n/a	n/a	n/a	n/a
bltu	input_pc	pcUp4 or pc+imm	RS1	RS2	RS1	RS2	n/a	n/a	n/a	n/a	n/a
bgeu	input_pc	pcUp4 or pc+imm	RS1	RS2	RS1	RS2	n/a	n/a	n/a	n/a	n/a
jal	input_pc	pc+imm	n/a	n/a	n/a	n/a	Pc+4	s_EXpcUp4	s_memALUWriteBack	s_wbALUWriteBack	s_RegWrData
jalr	input_pc	rs1+imm	rs1	imm	n/a	n/a	Pc + 4	s_EXpcUp4	s_memALUWriteBack	s_wbALUWriteBack	s_RegWrData
lb	input_pc	pcUp4	RS1	IMM	RS1	IMM	S_ALUresult = RS1 && IMM : RS2 passed to next stage	s_memALUWriteBack	s_memMemOut	s_wbMemOut	s_RegWrData
lh	input_pc	pcUp4	RS1	IMM	RS1	IMM	S_ALUresult = RS1 && IMM : RS2 passed to next stage	s_memALUWriteBack	s_memMemOut	s_wbMemOut	s_RegWrData
lbu	input_pc	pcUp4	RS1	IMM	RS1	IMM	S_ALUresult = RS1 && IMM : RS2 passed to next stage	s_memALUWriteBack	s_memMemOut	s_wbMemOut	s_RegWrData
lhu	input_pc	pcUp4	RS1	IMM	RS1	IMM	S_ALUresult = RS1 && IMM : RS2 passed to next stage	s_memALUWriteBack	s_memMemOut	s_wbMemOut	s_RegWrData
slli	input_pc	pcUp4	RS1	IMM	RS1	IMM	S_Bsresult = RS1 << IMM	s_memALUWriteBack	s_memALUWriteBack	s_wbALUWriteBack	s_RegWrData
srli	input_pc	pcUp4	RS1	IMM	RS1	IMM	S_Bsresult = RS1 >> IMM left bits replaced by 0's	s_memALUWriteBack	s_memALUWriteBack	s_wbALUWriteBack	s_RegWrData
srai	input_pc	pcUp4	RS1	IMM	RS1	IMM	S_Bsresult = RS1 >> IMM left bits replaced by sing bit	s_memALUWriteBack	s_memALUWriteBack	s_wbALUWriteBack	s_RegWrData
auipc	input_pc	pc+ imm	n/a	IMM	n/a	n/a	n/a	n/a	n/a	n/a	s_RegWrData
wfi											

Combined consuming and producing signals into 1spreadsheet.

[2.b.iii] generalized list of potential data dependencies. From this generalized list, select those dependencies that can be forwarded (write down the corresponding pipeline stages that will be forwarding and receiving the data), and those dependencies that will require hazard stalls.

There a a total of 7 different types of hazards that can occur for both RS1/RS2:

1. EX-to-EX Forwarding (MEM → EX)

This hazard occurs when the instruction in EX needs a value that the previous instruction is producing in MEM. The value is not yet written back, but the ALU result is already available. The processor forwards the MEM ALU output directly into the EX ALU inputs to avoid stalling.

2. MEM-to-EX Forwarding (WB → EX)

This hazard happens when an instruction in EX depends on a result currently in the WB stage. The register file does not yet contain the correct value because WB writes at the end of the cycle. Therefore, the WB result is forwarded directly into the EX stage ALU inputs.

3. Load-Use Hazard (Load in MEM → Use in EX)

A load-use hazard occurs when a load instruction is in MEM and the next instruction in EX needs the loaded value. Since load data is only ready at the end of MEM, it cannot be forwarded in time. This requires inserting a one-cycle stall so the load can complete.

4. Store-Load Hazard (WB Load → MEM Store RS2)

This hazard appears when a store instruction in MEM needs RS2 data that is being loaded by an instruction currently in WB. The register file does not yet contain the correct value. The processor must forward the WB load value to the store data input to prevent writing the wrong data to memory.

5. Control Hazard (EX → ID)

A branch or jump instruction in ID may need a register value that an earlier instruction is still computing in EX. Because the branch decision depends on these registers, ID cannot read the correct value yet. The pipeline stalls so the EX stage can finish producing the needed value.

6. Control Hazard (MEM → ID)

This hazard occurs when a branch or jump instruction in ID depends on a register that an earlier instruction is writing in MEM. The value is not yet available in the register file. The processor forwards the MEM result to ID stage.

7. Register File Bypass (WB → ID)

When an instruction in ID reads a register that is being written by the current WB stage, it would normally read an outdated value. To prevent this, the processor bypasses the WB data directly to the ID stage operand path. This avoids a stall and ensures the instruction receives the updated value immediately.

[2.b.iv] global list of the datapath values and control signals that are required during each pipeline stage

IF STAGE:
IF register signals (going into ID register):

- s_IDpc,
- s_idInst : std_logic_vector(31 downto 0);

Fetch Unit Signals

- s_IDpcUp4 : std_logic_vector(31 downto 0);
- s_IdpcUpImm : std_logic_vector(31 downto 0);
- s_RegPlusImm : std_logic_vector(31 downto 0);
- s_branchLogic : std_logic;
- s_idBranchOrJump : std_logic;

[2.c.i] list all instructions that may result in a non-sequential PC update and in which pipeline stage that update occurs.

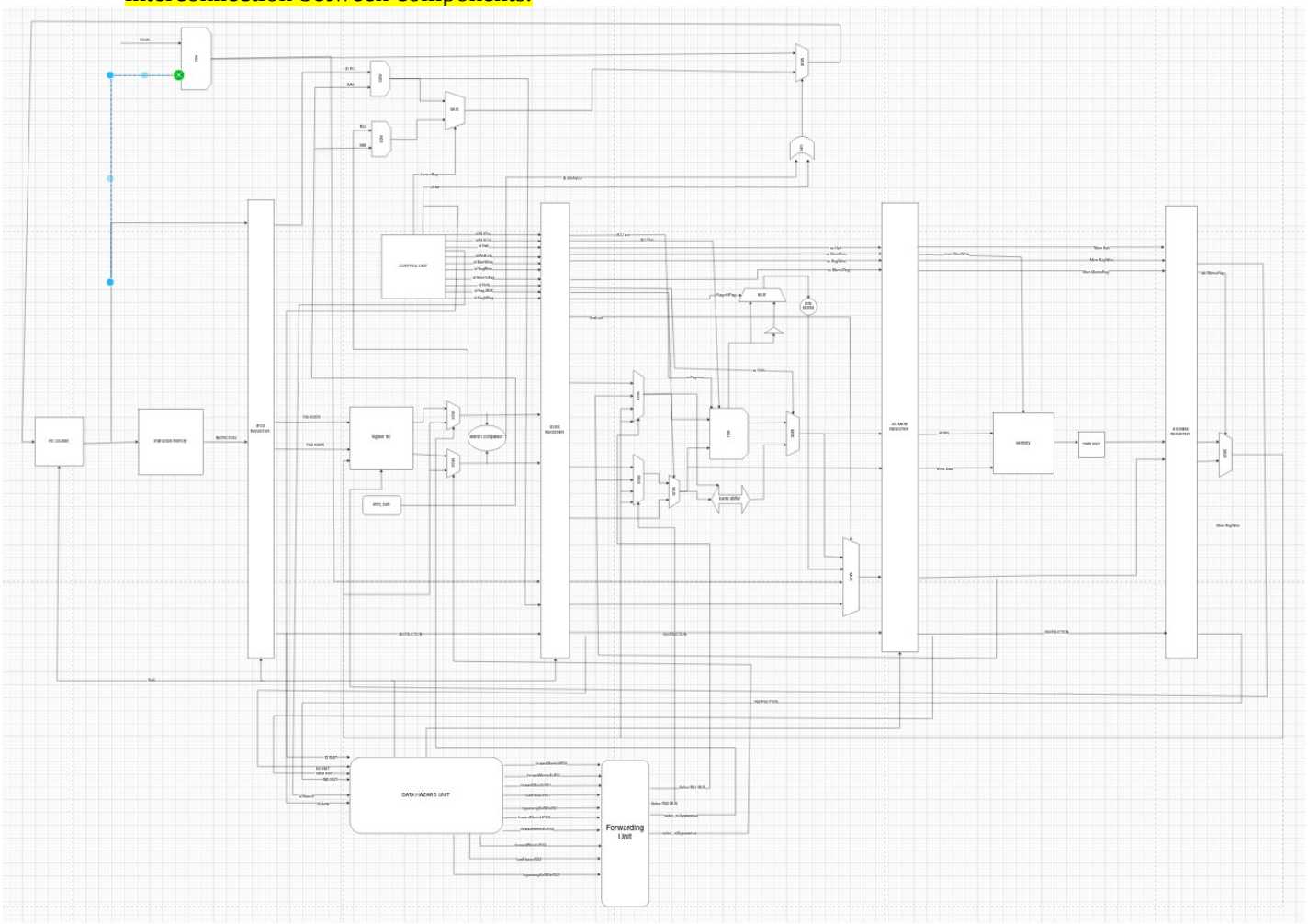
BEQ, BNE, BGE, BLT, BLTU, BGEU, JAL, JALR

[2.c.ii] For these instructions, list which stages need to be stalled and which stages need to be squashed/flushed relative to the stage each of these instructions is in.

For our implementation, when a control hazard occurs, we have to stall the IF stage by setting the IF/ID and PC registers' write enable lines to 0.

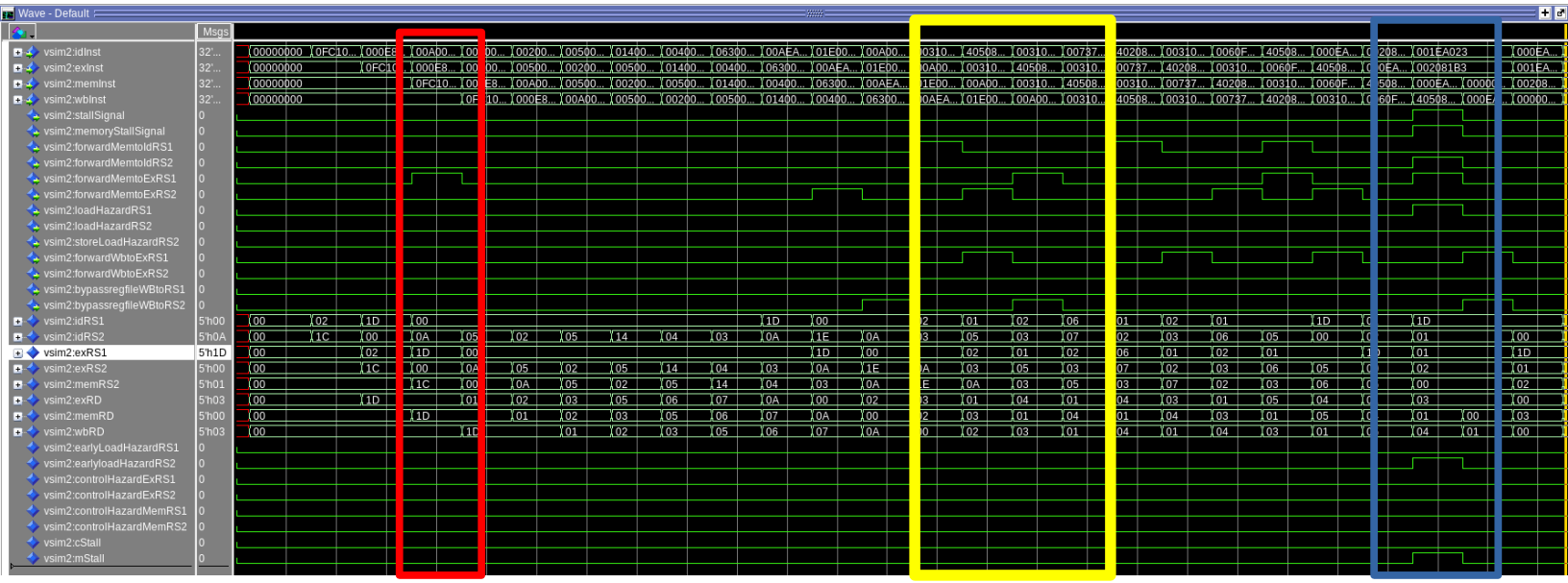
Since we calculate all our non-sequential PC updating instructions in the ID stage, we have eliminated the need to squash or flush any stages.

[2.d] implement the hardware-scheduled pipeline using only structural VHDL. As with the previous processors that you have implemented, start with a high-level schematic drawing of the interconnection between components.



[2.e – i, ii, and iii] In your writeup, show the QuestaSim output for each of the following tests, and provide a discussion of result correctness. It may be helpful to also annotate the waveforms directly.

Data Hazard Tests



Red: Regular Forwarding Event

This regular forwarding event resulted from the instruction in the memory stage writing to register x28. The next instruction in the sequence uses x28 as the RS1 value. By comparing the mem-stage RD and the ex-stage RS1, it triggered a forwarding signal, which we then sent to the forwarding unit to switch our input A multiplexer to the memory-forwarded value.

Yellow: Consecutive Forwarding Events

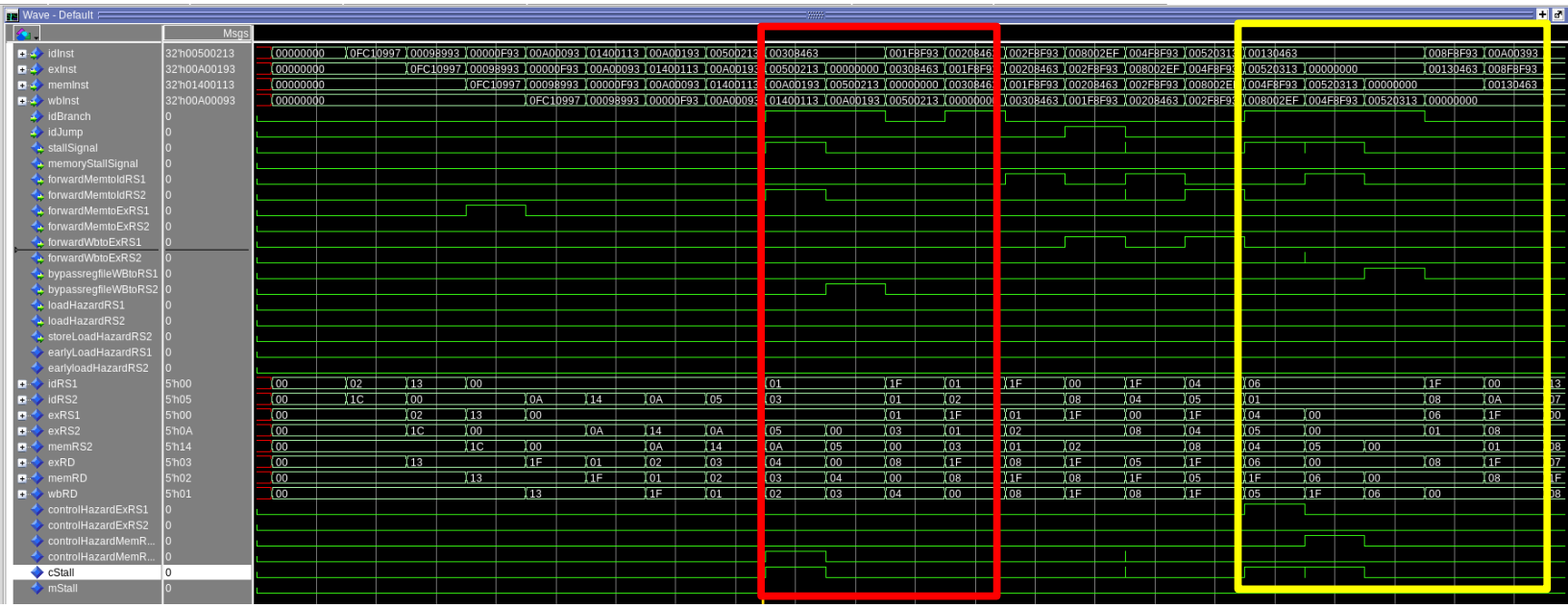
This sequence of 4 instructions caused 5 forwarding events to happen:

MEM → ID
MEM → EX
WB → EX (redundant)
MEM → EX (2)
WB → ID (bypassing)

This covers a ton of events that need to be checked over and ensuring that these events trigger properly is important for coverage.

Blue: Load-Use Event Causing a Stall

One of the unique cases that will always necessitate a stall cycle is the Load-Use case, where the sequence of instructions includes a LOAD → USAGE pattern, whether that is LW x1… ADD x1, x1, x1, or another following instruction. Because of the need to pull the ‘loaded’ value out of memory, the load instruction must reach the WB stage. For this to happen, particularly when there is a directly following instruction using that register, the IF, ID, and EX stages need to be stalled.



Control Hazard Tests

Red: Single stall for branch dependency

The first case is a simple forwarding during a branch. Since branches use RS1/2, we can fall back on our forwarding logic from Data Hazards, and expand the cases to forward back to the ID stage and cover MEM → ID and WB → ID. Since we work in the ID stage for our branch/jump logic, we have to stall the dependency out of the EX stage to properly forward.

Yellow: Load-Branch Consecutive, 2 stalls

A similar case to the Load-Use situation with Data Hazards, we have to stall 2 times in order to branch using a being-loaded register. We have to stall the load dependencies into the WB stage in order to properly use them.

[2.e.i] Create a spreadsheet to track these cases and justify the coverage of your testing approach. Include this spreadsheet in your report as a table.
[2.e.ii] Create a spreadsheet to track these cases and justify the coverage of your testing approach. Include this spreadsheet in your report as a table.

Hazard Type	Example	Forwarding Path	Stall Cycles	Flushed Stages	Notes and Coverage
Data Hazards					
Read After Write (RAW)	ADD x1, x2, x3 SUB x4, x1, x5	MEM → EX	0	None	The RAW errors are extremely common and are covered multiple times in many formats. Since the I-type and R-type instructions use the same RS1/RS2/Immediate structures only 1 of each is necessary.
RAW – Non Consecutive	ADD x1, x2, x3 AND x4, x6, x7 SUB x3, x1, x2	WB → EX	0	None	We cover these in consecutive cases (1–5) in our Data Hazards Tests
Cascading RAW Hazards	ADD x1, x2, x3 AND x5, x1, x6 SUB x4, x1, x5	MEM → EX MEM → EX	0	None	
Load-Use	LW x1, 0(x2) ADD x3, x1, x2	WB → EX	1	None	
Store-Load	SW x1, 0(x2) LW x3,0(x2)	WB → MEM	0	None	
Control Hazards					
Branch/Jump Consecutive	ADD x1, x2, x3 BNE x1, x2, target	EX → ID	1	None	Due to our implementation, we do not require any flushing on a branch or jump taken
Branch/Jump Non-Consecutive 1	ADD x1, x2, x3 ADD x4, x4, x4 JAL x1, target	MEM → ID	0	None	All branches and jumps share forwarding and stalling implementations
Branch/Jump Non-Consecutive 2	ADD x1, x2, x3 ADD x4, x4, x4 ADD x5, x5, x5 JALR x1, 0(x2)	WB → ID	0	None	Due to not requiring any flushing, our processor has no consecutive branch/jump hazards.
Branch/Jump After Load Consecutive	LW x1, 0(x2) BNE x1, x3, target	WB → ID	2	None	We cover the simple branch taken/not taken and then progress in our testing to cover more situations and detect the stalls needed to pass the branch.
Branch/Jump After Load Non-Consecutive 1	LW x1, 0(x2) AND x4, x6, x7 JALR x1, 0(x9)	WB → ID	1	None	
Branch/Jump After Load Non-Consecutive 1	LW x1, 0(x2) AND x4, x6, x7 AND x5, x6, x7 JALR x1, 0(x9)	WB → ID	0	None	

[2.f] report the maximum frequency your hardware-scheduled pipelined processor can run at and determine what your critical path is (specify each module/entity/component that this path goes through).

31.20MHz

Critical Path: (JALR Instruction)
Imem → Register Bypass Multiplexer → Adder for JALR → FetchUnit (Updating PC register)